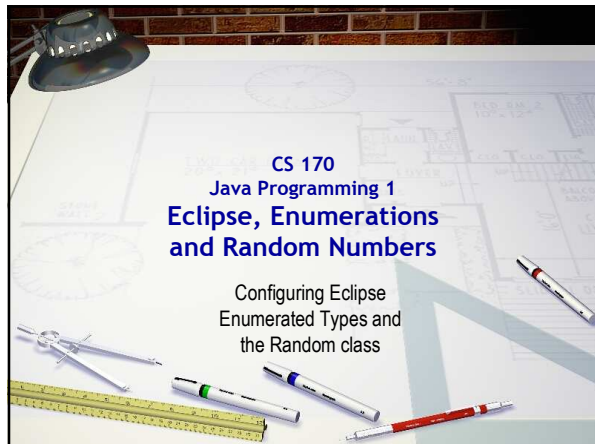
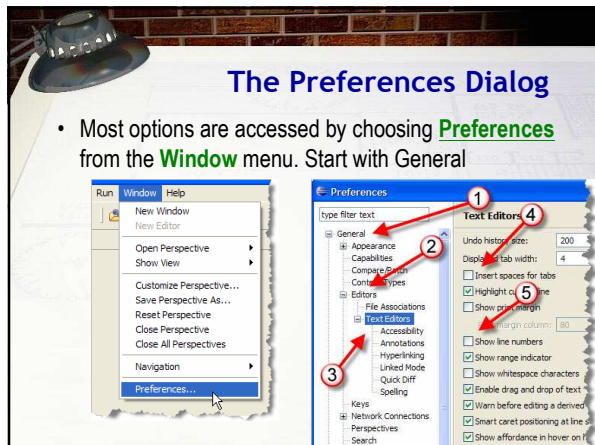
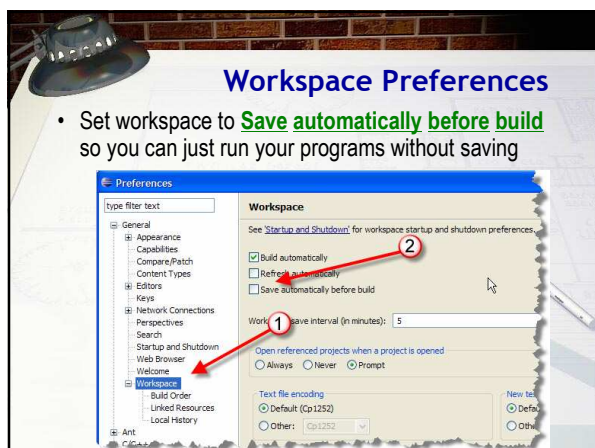


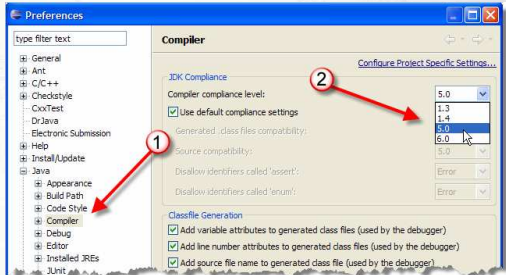






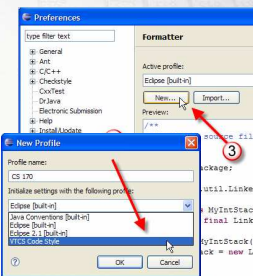
Compiler Conformance

- Used for generated code. Set to 5.0 for compatibility.



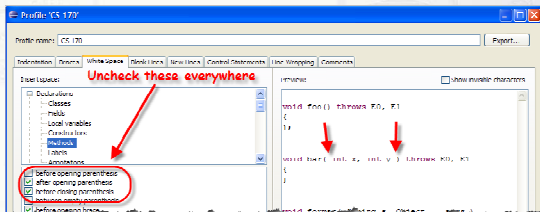
Code Formatting

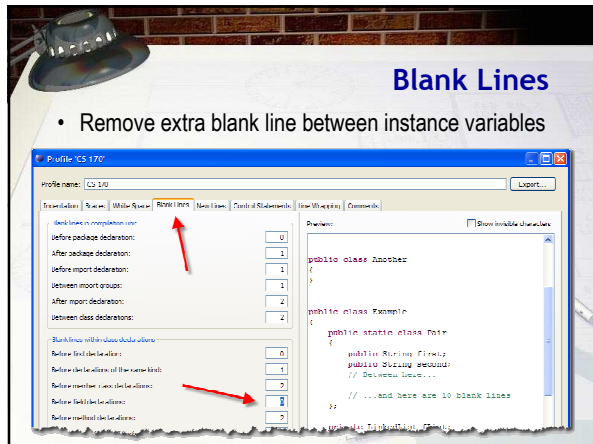
- Eclipse will automatically **format** your code for you
 - Built-in styles are different than class or text
- Create a formatting profile for CS 170
 - Choose Formatter under Code Style
 - Click New
 - Name the profile CS 170
 - Base it on VTCS style

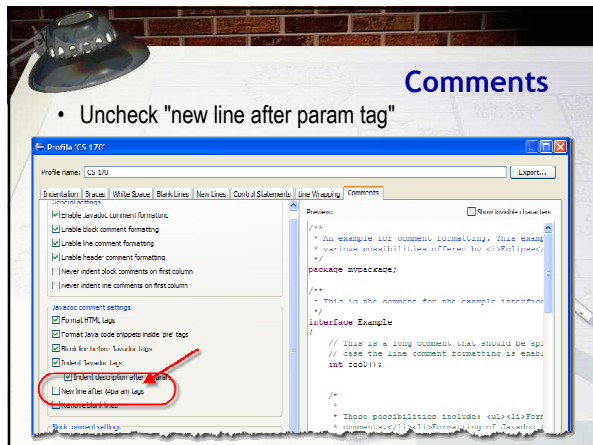


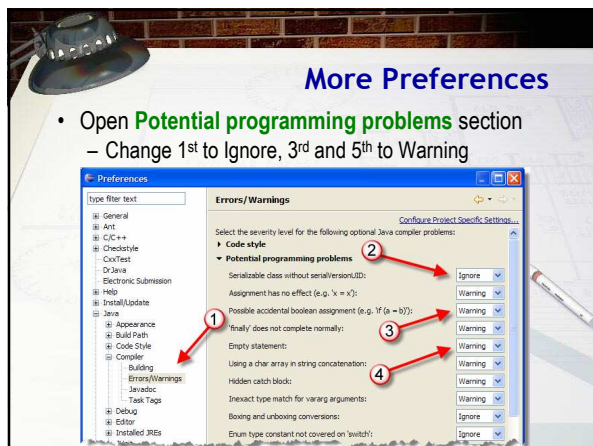
Whitespace

- Remove extra whitespace in function calls, expressions and control statements
 - Remove "after opening parentheses" and "before closing parentheses" in control statements, etc.



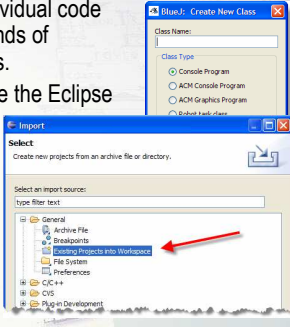






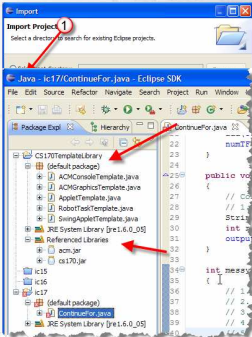
A Template Project

- Eclipse doesn't have individual code templates for different kinds of programs like BlueJ does.
- Let's see how we can use the Eclipse **Import Project** function to create a library of templates.
- Choose **File->Import** then **Existing Projects into Workspace**



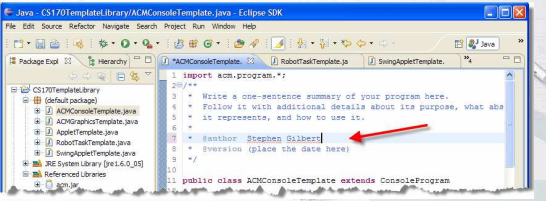
Projects from Archives

- Click **Select Archive file**, then **Browse** button
- Locate and select the file: **CS170TemplateLibrary.zip** on the Q: drive then click the **Finish** button
- Exercise 1:** Expand default package and Referenced Libraries like this and snap a screen-shot.



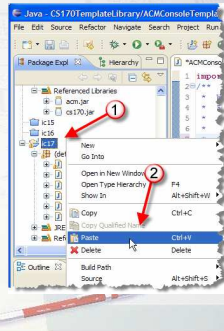
Adding Comments

- When you use a template file, Eclipse won't be able to add the custom comments that you set up
 - No problem, you can just add them to the templates
- Exercise 2:** Open each template file, add your name



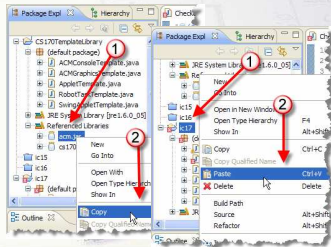
Projects from Templates

- Let's create an ACM Graphics program with nested loops.
- Create a new project for this week: **ic15**
 - Select and then right-click **ACMGraphicsTemplate**, choose **Copy** from the menu.
 - Then, select the **ic15** project and choose **Paste**



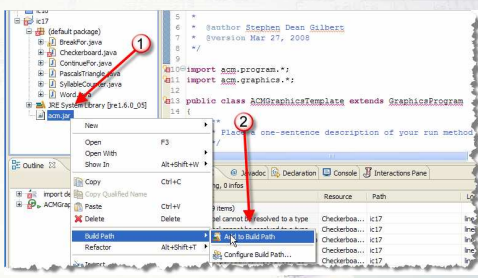
Adding a Library File

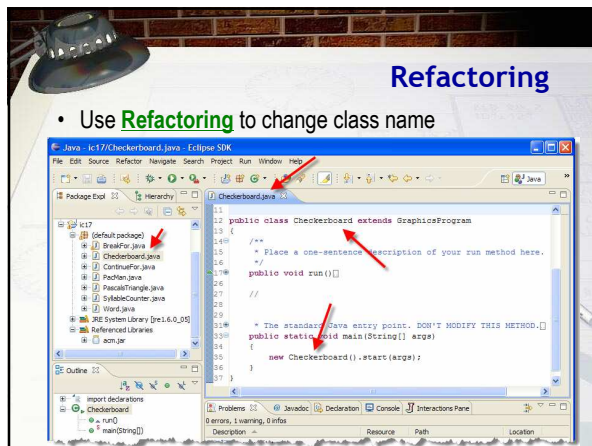
- Code doesn't compile; there are **two** problems
 - Need the library file, **acm.jar** in the project
 - Need the class name to match the file name
- Right-click **acm.jar** in the Referenced libraries section of the template project and choose **Copy**
- Right-click **ic15** and click **Paste**



Adjusting the Build Path

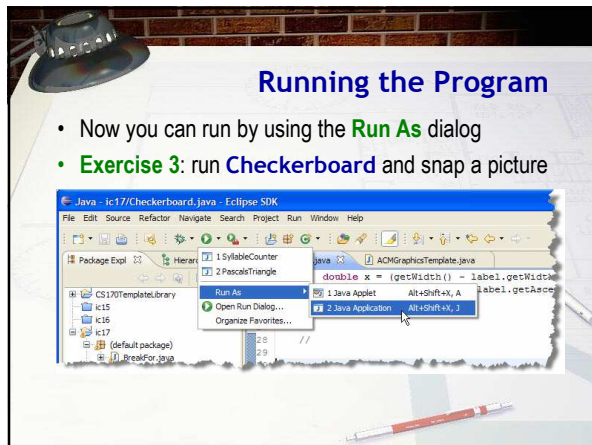
- Locate **acm.jar** in **ic15**, and right-click the file
 - Select **Build Path->Add to Build Path** and save





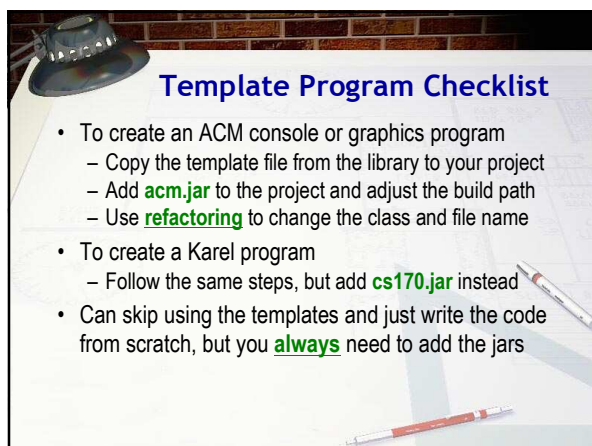
Refactoring

- Use **Refactoring** to change class name



Running the Program

- Now you can run by using the **Run As** dialog
- **Exercise 3:** run **Checkerboard** and snap a picture



Template Program Checklist

- To create an ACM console or graphics program
 - Copy the template file from the library to your project
 - Add **acm.jar** to the project and adjust the build path
 - Use **refactoring** to change the class and file name
- To create a Karel program
 - Follow the same steps, but add **cs170.jar** instead
- Can skip using the templates and just write the code from scratch, but you **always** need to add the jars

Exercise 4: Run and snap a picture.

Finishing Checkerboard

- Let's use nested loops to draw the checkerboard
 - Erase the code inside the `run()` method and then
 - Add a constant `ROWS` set to **8**
 - Add a **double** variable for the size of each square
 - Initialize by using `getHeight()` and dividing by `ROWS`
 - Write a nested **for** loop:
 - Outer loop from `0 < ROWS`, inner from `0 < ROWS`
 - Calculate **x** (inner * square size) and **y** (outer * square size)
 - Create a new `GRect` using **x**, **y** and the square size
 - If outer + inner are odd, then `setFilled(true)`
 - Add the `GRect` to the program surface

Card Demo

- Last two topics left to cover in Chapter 5 and 6 are **enumerated types** and the `Random` class.
- Let's return to our playing card example and create a class that represents playing cards
- Exercise 5:** Copy the cards folder and add it to your project. Then import the `CardDemo` program. Shoot a screenshot of your workspace.

CardDemo.java

- `CardDemo` creates a stack of `Card` objects on table
 - Some parts are missing. Let's start with `Card`.

```
CardDemo.java
add(table);
// Figure out where the deck of cards should go
Card c = new Card(Rank.TWO, Suit.CLUBS);
GPoint loc = new GPoint((getWidth() / 2.0 - c.getWidth() / 2.0,
                        (getHeight() / 2.0 - c.getHeight() / 2.0));

// Add all the cards to the deck
for (Suit suit: Suit.values())
{
    for (Rank rank: Rank.values())
    {
        c = new Card(rank, suit);
        add(c, loc);
    }
}

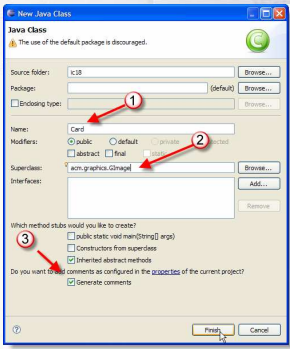
// Listen for mouse events
```

Card class undefined

Each card has a Suit and a Rank

The Card Class

- Create a new class **Card**, **acm.graphics.GImage**, no main method, comments



Modeling Playing Cards

- Every playing card has two defining data elements
 - Rank**: 2-10, Jack, Queen, King, Ace
 - Suit**: Clubs, Spades, Diamonds, Hearts
- We'll represent these as instance variables
 - The question is, what kind of data?
 - String? Assume we have a constructor like this:
 - Card(String rank, String suit);**
 - User writes **Card c = new Card("Ac", "Cubs");**
- Compiler can't catch this, but **enumerated types** can

Enumerations

- Enumerations are user-defined types like classes
 - Each type has a restricted number of values
 - Think days of the week, months, compass points, etc.
- Create an enumerated type using the **enum** keyword
 - These types are **strongly type-checked**
 - A **better** way to create named constants
- Exercise 6:** In Eclipse, create two new **Enums**, one named **Rank** and one named **Suit**. Shoot a screenshot of your workspace.



Specifying Values

- Need a name for each value. Since they are constant, its convention to use upper-case for the name

```

10 /**
11  * Suit.java: Playing-card suits.
12  *
13  * @author Stephen Dean Gilbert
14  * @version Mar 28, 2008
15  */
16 public enum Suit
17 {
18     CLUBS, SPADES, DIAMONDS, HEARTS;
19 }

```

Comma separated list of names.

Exercise 7: Add values for Rank. Start at TWO, end with ACE.

Using Enumerated Types

- Use them like an "integer" wrapper type

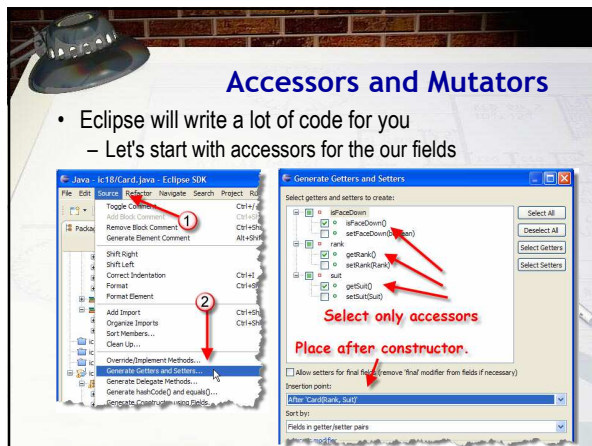
```

Suit suit;           // an enum variable
suit = Suit.HEARTS;  // initializing
suit == Suit.CLUBS;  // compare with != and ==
switch(suit) {       // use as a switch selector
    suit.ordinal()    // position in list
    Suit.values().length // elements in enum list
    Suit.values()[n]  // elements number n
    Card(Rank rank, Suit suit) // formal parameters
}

```

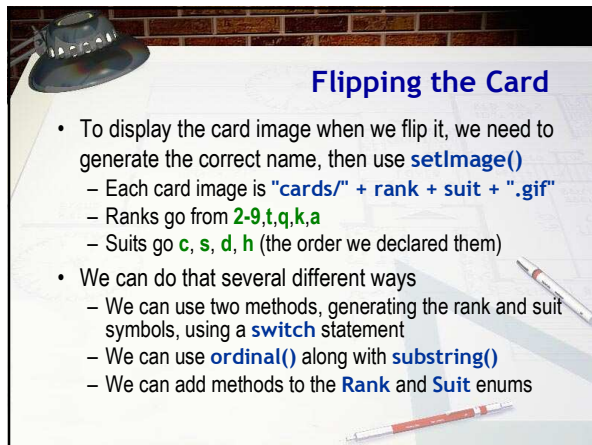
Constructing Cards

- Let's add our fields and a constructor to **Card**
 - Add a **private Rank** and **Suit** instance variable
 - Create a constructor that takes a **Rank** and **Suit**
 - Use **super("cards/b.gif");** as the first line
 - This initializes the **GImage** part using the card back
 - Assign both parameters to the fields you defined
- Everything now compiles except one missing method
 - Add a **private boolean** field, **isFaceDown**
 - Add a void method **flip()** that just toggles the field
- Exercise 8: run and shoot a screen-shot.



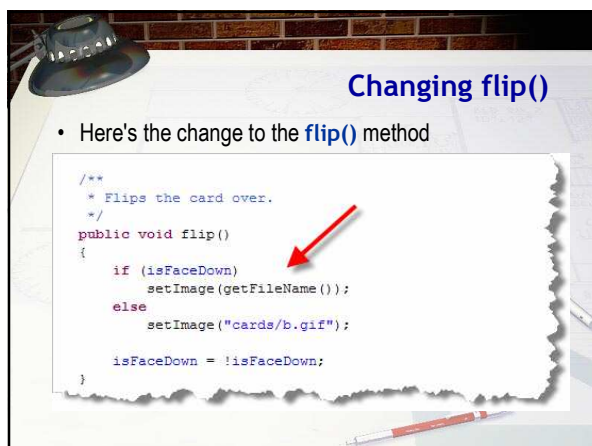
Accessors and Mutators

- Eclipse will write a lot of code for you
 - Let's start with accessors for the our fields



Flipping the Card

- To display the card image when we flip it, we need to generate the correct name, then use `setImage()`
 - Each card image is "cards/" + rank + suit + ".gif"
 - Ranks go from 2-9,t,q,k,a
 - Suits go c, s, d, h (the order we declared them)
- We can do that several different ways
 - We can use two methods, generating the rank and suit symbols, using a `switch` statement
 - We can use `ordinal()` along with `substring()`
 - We can add methods to the `Rank` and `Suit` enums



Changing flip()

- Here's the change to the `flip()` method

```
/**
 * Flips the card over.
 */
public void flip()
{
    if (isFaceDown)
        setImage(getFileName());
    else
        setImage("cards/b.gif");

    isFaceDown = !isFaceDown;
}
```

Getting the File Name

- Here's the `getFileName()` method:

```
/**
 * Calculates the file-name for this card.
 */
private String getFileName()
{
    return "cards/" + rankSymbol() + suitSymbol() + ".gif";
}
```

The suitSymbol() Method

- Calculates the symbol based on its **ordinal** position
 - Real drawback:** breaks if you change order in enum

```
/**
 * Returns the suit symbol.
 */
private String suitSymbol()
{
    int pos = suit.ordinal();
    return "csdh".substring(pos, pos+1);
}
```

Position in enum definition

Only works if this is in-synch

The rankSymbol() Method

- Uses **switch**, so still works if values are reordered in enum definition
- Exercise 9:** Run, make sure you can drag and flip the cards. Snap a pic.

```
/**
 * Calculates the rank symbol using a switch.
 */
private String rankSymbol()
{
    switch(rank)
    {
        case TWO:    return "2";
        case THREE:  return "3";
        case FOUR:   return "4";
        case FIVE:   return "5";
        case SIX:    return "6";
        case SEVEN:  return "7";
        case EIGHT:  return "8";
        case NINE:   return "9";
        case TEN:    return "t";
        case JACK:   return "j";
        case QUEEN:  return "q";
        case KING:   return "k";
        default:     return "a";
    }
}
```

Random Numbers with Math

- The **Math.random()** method is very easy to use
 - Generates random numbers between 0.0 and 1.0
 - 0.0 is included and 1.0 is excluded
- Normally, we want a wider range and a different type
 - Use multiplication to get the "number of instances"
 - Use casting to convert to the correct type
 - Use addition to put the number in the range you want

```
int die = (int) (Math.random() * 6) + 1;
```

Let 'em Roll

- Imagine a lottery game where winner must guess six numbers between 39 and 173 (inclusive)
 - Here's how you'd generate the first number in the sequence

```
int min = 39;  
int max = 173;  
int range = max - min + 1;  
int p1 = (int) (Math.random() * range) + min;
```



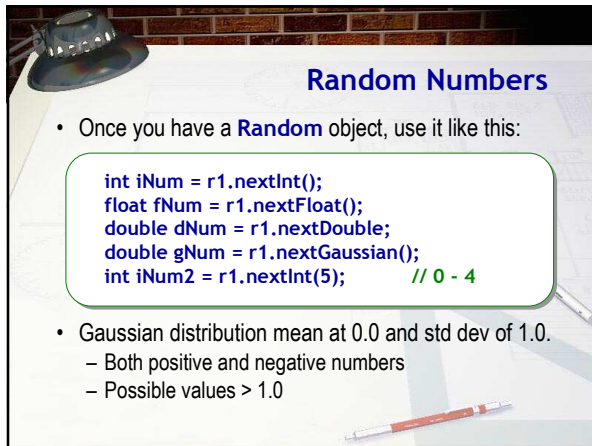
- Exercise 10:** Generate in Dr. Java. Screen shot.

The Random Class

- The **java.util.Random** class provides a more complete set of random number generating functions
 - Unlike **Math.random()**, these are not static
 - You first create a **Random** object like this:

```
Random gen1 = new Random();  
Random gen2 = new Random(20L);
```

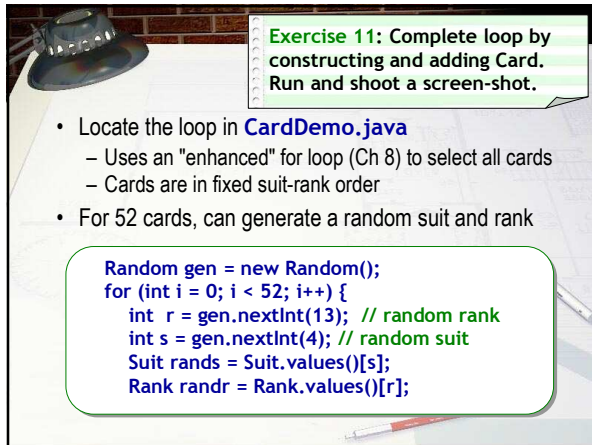
- The 20L passed to the second constructor is a **seed**
- Same sequence of numbers each time it is used
- Default constructor uses the system clock for the seed



Random Numbers

- Once you have a **Random** object, use it like this:

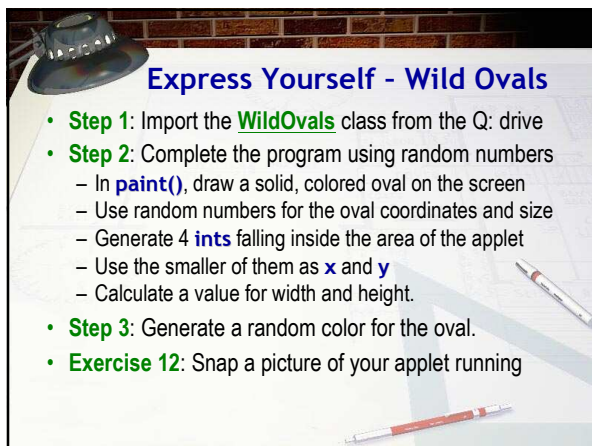
```
int iNum = r1.nextInt();
float fNum = r1.nextFloat();
double dNum = r1.nextDouble();
double gNum = r1.nextGaussian();
int iNum2 = r1.nextInt(5);    // 0 - 4
```
- Gaussian distribution mean at 0.0 and std dev of 1.0.
 - Both positive and negative numbers
 - Possible values > 1.0



Exercise 11: Complete loop by constructing and adding Card. Run and shoot a screen-shot.

- Locate the loop in **CardDemo.java**
 - Uses an "enhanced" for loop (Ch 8) to select all cards
 - Cards are in fixed suit-rank order
- For 52 cards, can generate a random suit and rank

```
Random gen = new Random();
for (int i = 0; i < 52; i++) {
    int r = gen.nextInt(13); // random rank
    int s = gen.nextInt(4); // random suit
    Suit rands = Suit.values()[s];
    Rank randr = Rank.values()[r];
}
```



Express Yourself - Wild Ovals

- Step 1:** Import the **WildOvals** class from the Q: drive
- Step 2:** Complete the program using random numbers
 - In **paint()**, draw a solid, colored oval on the screen
 - Use random numbers for the oval coordinates and size
 - Generate 4 **ints** falling inside the area of the applet
 - Use the smaller of them as **x** and **y**
 - Calculate a value for width and height.
- Step 3:** Generate a random color for the oval.
- Exercise 12:** Snap a picture of your applet running
